

C# Einführung

Kursablauf – Agenda

Agenda 1

- ★ 01 Kennenlernen
- ★ 02 Einführung
- ★ 03 C# und .NET
- ★ 04 Erste Schritte in C#
- ★ 05 Klassen und Objekte
 - Übung
- ★ 06 Grundlagen Datentypen
 - Übung
- ★ 07 Ablaufsteuerung
 - Übung

Agenda 2

- ★ 08 Operatoren
- ★ 09 Erweiterte Datentypen
 - Übung
- ★ 10 Vererbung und Interfaces
 - Übung
- ★ 11 Eigenschaften und Indexer
 - Übung
- ★ 12 Strukturierte Fehlerbehandlung

Agenda 3

- ★ Wie weiter?
- ★ Optional (falls noch Zeit)
 - Ausgewähltes Thema

Einführung

- ★ C# was ist das?
- ★ C# ist die Systemsprache für das .NET Framework
- ★ C# baut auf C++ bzw. Java auf
- ★ Ist eine rein objektorientierte Programmiersprache

Nutzen des Kurses

- ★ Sie kennen und können einsetzen:
 - Elementare Bestandteile der Sprache C#
 - Entwicklung eigener kleiner Programme
 - Umsetzung eines einfachen OO Designs in C#
- ★ Sie können sich nach dem Kurs selbständig in fortgeschrittene Themen von C# einarbeiten

C# und .NET

Ziele

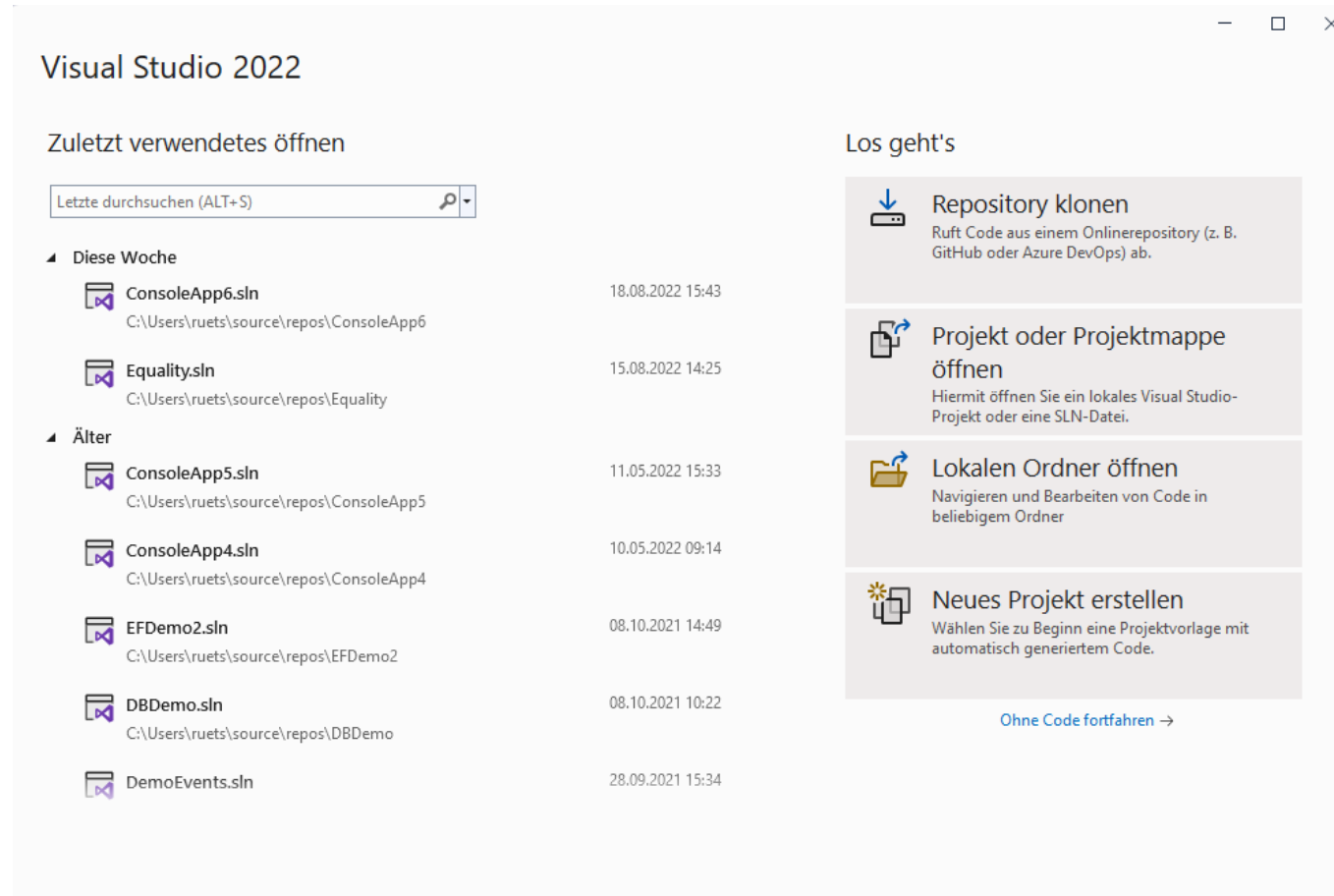
- ★ Zusammenhang zwischen C# und .NET kennen
- ★ Die Wichtigkeit von C# für .NET einschätzen können
- ★ Sie kennen die Hauptelemente des .NET Frameworks

Erste Schritte in C#

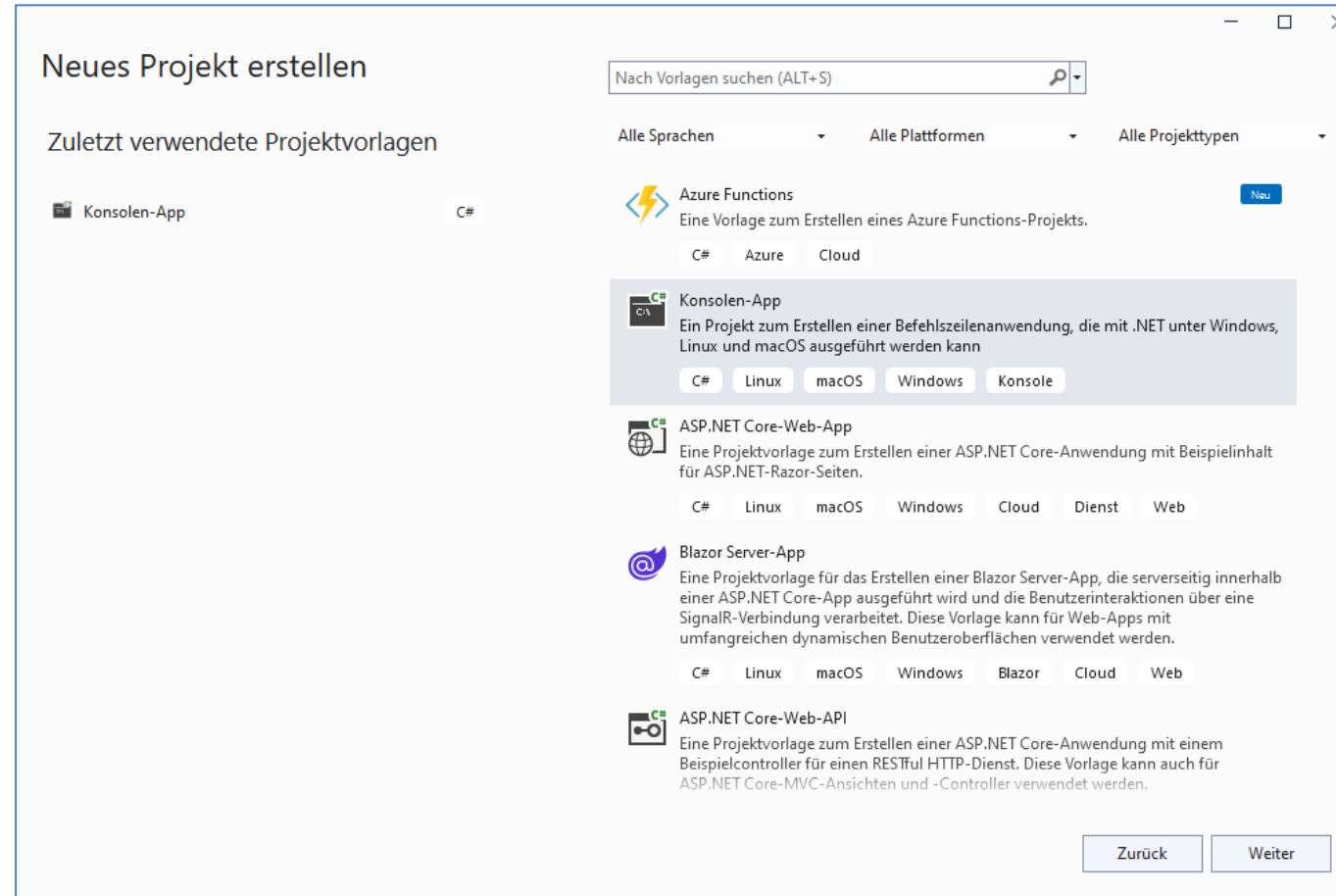
Lernziel

- ★ Sie können Visual Studio .NET benutzen, um ein «Hello World»-Programm in C# zu schreiben und Sie verstehen dessen wesentlichen Elemente.
 - Blöcke
 - Kommentare
 - Die Main() Methode
 - Namensräume (Namespaces)
- ★ Erstellen einer Konsolen-Applikation mit Visual Studio .NET
- ★ Bestandteile eines C#-Programms
- ★ Hilfesystem des Visual Studio.NET

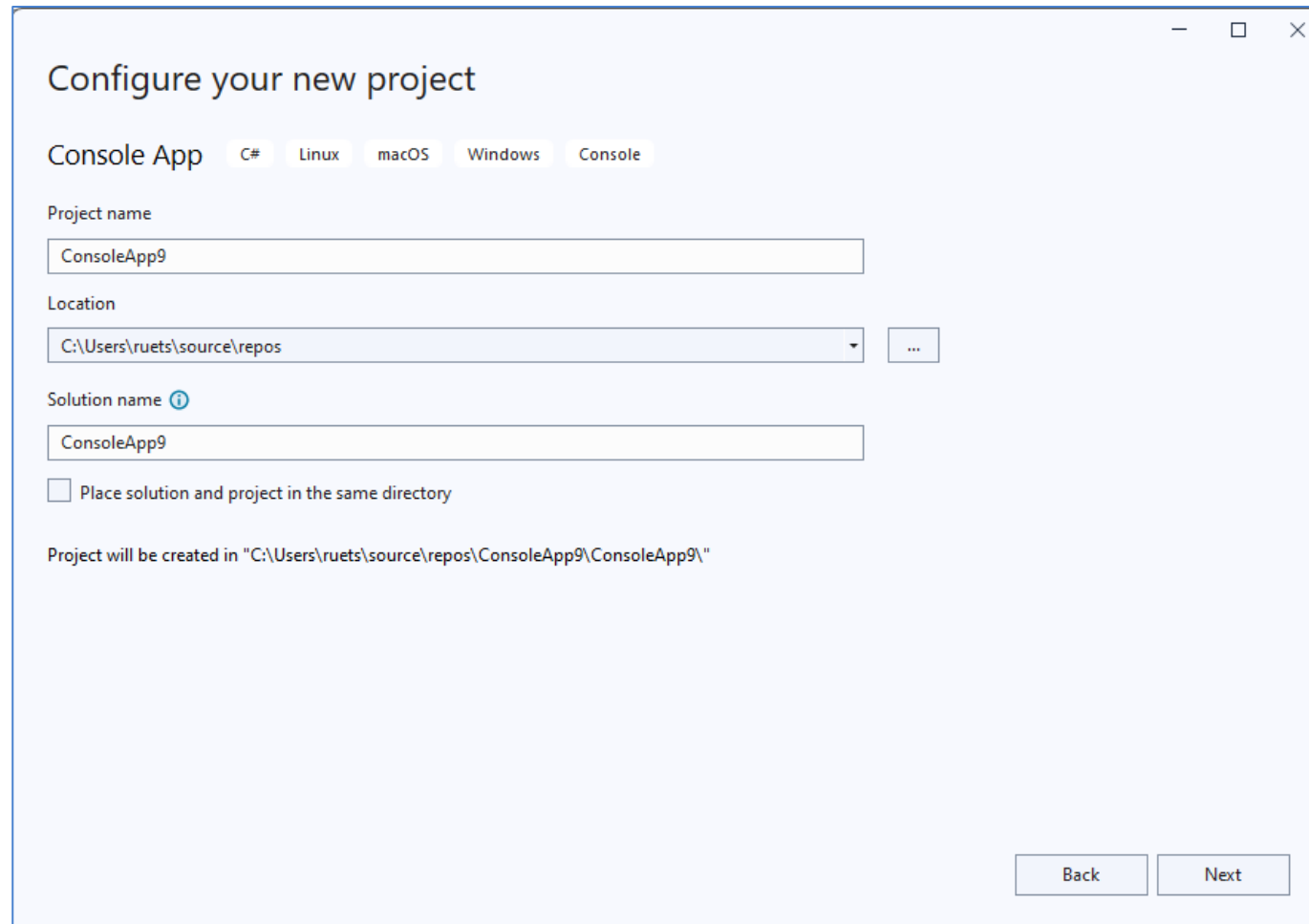
Startfenster



Die VS-Umgebung



New Project Dialog



Configure your new project

Console App C# Linux macOS Windows Console

Project name
ConsoleApp9

Location
C:\Users\ruets\source\repos

Solution name ⓘ
ConsoleApp9

☐ Place solution and project in the same directory

Project will be created in "C:\Users\ruets\source\repos\ConsoleApp9\ConsoleApp9\"

Back Next

Erstes Programm

- ★ Demo «Hello World»
- ★ Kontrollfragen

Klassen und Objekte

Lernziel

★ Sie kennen das wichtigste Konstrukt objektorientierter Programmiersprachen: die Klasse

- Klassen und Objekte
- Felder einer Klasse
- Methoden einer Klasse
- Namensräume

Begriffe

- ★ Klasse («Bauplan» eines Objekts)
- ★ Objekt (Instanz einer Klasse)
- ★ Member (Eigenschaften und Methoden einer Klasse)
- ★ Eigenschaften (definieren Zustand)
- ★ Methoden (definieren Verhalten)

Deklaration einer Klasse + Erzeugen einer Instanz

```
namespace Garage
{
    public class Vehicle
    {
        private string vehicleName;

        public Vehicle(string name) // Constructor
        {
            vehicleName = name;
        }

        public static void Main(string[] args)
        {
            Vehicle v1 = new Vehicle("car");
            Vehicle v2 = new Vehicle("bike");
        }
    }
}
```

3 Arten von Variablen

- ★ Instanzvariablen
 - ★ Statische Variablen / Klassenvariablen
 - ★ Lokale Variablen
-
- ★ Hinweis: Instanz- und Klassenvariablen sind beides Felder

1. Felder einer Klasse (1/3)

★ Syntax

```
[Modifikator] Datentyp Bezeichner [=Initialwert];
```

★ Beispiele

```
public string firstName = "Frank";
```

```
private int nrOfEntry = 1;
```

✓ Felder definieren den Zustand eines Objekts

Regeln für Bezeichner (2/3)

- ★ Bezeichner dürfen mit einem Buchstaben oder einem «_» (Unterstrich) beginnen
- ★ Innerhalb des Bezeichnernamens dürfen auch Zahlen verwendet werden, nicht aber am Anfang
- ★ Bezeichner dürfen nicht aus mehreren Worten bestehen

★ Beispiele:

Gültige Bezeichner:

- myName
- _theName
- x1
- Name5S7

Ungültige Bezeichner:

- 1stStart
- Mein Name
- &again

Felder (3/3)

★ Instanzvariablen

- Normale Felder einer Klasse
- Sie werden Instanzvariablen genannt, weil sie erst verfügbar sind, wenn eine entsprechende Instanz der Klasse angelegt worden ist
- Normalerweise private deklariert

```
public class Rectangle
{
    private int dx;
    private int dy;
}
```

2. Statische Variablen

★ Klassenvariablen

- Auch statische Variablen genannt, weil sie mit dem Modifikator `static` angelegt werden
- Verfügbar, wenn innerhalb des Programms der Zugriff auf die Klasse sichergestellt ist. Dies bedeutet: Es muss keine Instanz der Klasse geben.

```
public class Application
{
    public static Application theApplication;
    private Application() // private constructor
    {}
}
```


3. Lokale Variablen

- ★ Innerhalb eines durch geschweifte Klammern bezeichneten Programmblock deklariert
- ★ Ggelten lokal in diesem Programmblock, das heisst: Es kann nur in diesem Bereich auf sie zugegriffen werden
- ★ Bei jedem Aufruf des Programmblocks werden lokale Variablen neu initialisiert

```
public void Foo()  
{  
    int localVar = 0;  
}
```

Methoden einer Klasse

- ★ Methoden stellen die Funktionen einer Klasse dar

Syntax: **[Modifikator] Ergebnistyp Bezeichner([Parameter[,Parameter]])**
{
 // Anweisungen
}

```
using System;
```

```
public class Rectangle  
{  
    private int dx;  
    private int dy;  
  
    public int Area()  
    {  
        return dx * dy;  
    }  
    ...  
}
```

Methoden bestimmen das **Verhalten** einer Klasse

this

- ★ Die Referenz auf die aktuelle Instanz einer Klasse.

```
using System;
```

```
public class TestClass
```

```
{
```

```
    private int x = 10;
```

```
    public void DoOutput()
```

```
{
```

```
    int x = 5;
```

```
    Console.WriteLine("x hat den Wert {0}.", this.x);
```

```
}
```

```
}
```

Parameterübergabe

- ★ An Methoden können Parameter übergeben werden, die sich innerhalb der Methode wie lokale Variablen verhalten.
- ★ Es wird unterschieden zwischen:
 - Wertparametern (call by value)
 - Referenzparametern (call by reference)

Werteparameter

- ★ Mit Werteparametern werden Werte an eine Methode übergeben, die in dieser benutzt werden können, ohne die Variablen der aufrufenden Methode zu verändern
- ★ In der aufgerufenen Methode werden Kopien für die Variablen angelegt

```
public double Sqrt(double x)  
{  
    ...  
}
```

Referenzparameter

- ★ Deklariert durch das reservierte Wort `ref`
- ★ Es wird kein Wert, sondern eine Referenz auf die Variable des Aufrufers übergeben. Alle Änderungen an der Variablen innerhalb der aufgerufenen Methode ändern auch die Variable in der aufrufenden Methode
- ★ Die Variablen müssen vor dem Methodenaufruf initialisiert werden
- ★ Die aufgerufene Methode hat keine Kopie

★ Beispiele:

```
public void Swap(ref int a, ref int b)
{
    int c = a;
    a = b;
    b = c;
}
```

```
public void IsBigger(int a, int b, ref bool isOk)
{
    isOk = a > b;
}
```

Aufruf

```
IsBigger(5, 7, ref isOk);
```

out-Parameter

- ★ Funktionieren wie ref-Parameter, müssen jedoch im Gegensatz zu diesen vorher nicht initialisiert werden, eine allfällige Initialisierung wird verworfen.

```
using System;
class TestClass
{
    public static void IsBigger(int a, int b, out bool isOk)
    {
        // Erste Zuweisung = Initialisierung
        isOk = a > b;
    }
}
```

Optionale Parameter

★ Deklaration

```
public void Calculate(int x, int y = 5, int z = 7) { ... }
```

★ Mögliche Aufrufe

```
Calculate(1, 2, 3); // normaler Aufruf der Methode
```

```
Calculate(1, 2); // z weglassen - äquivalent zu Calculate(1, 2, 7)
```

```
Calculate(1); // y+z weglassen z - äquivalent zu Calculate(1, 5, 7)
```

★ Explizite Parameter

```
Calculate (1, z: 3); // Übergabe von z mit Namen
```

★ Risiko: optionale Parameter werden zur Compile-Zeit eingesetzt. Falls Sie später geändert werden, müssen abhängige Assemblies neu kompiliert werden

Überladen von Methoden

★ Mehrere Methoden haben den gleichen Namen

```
public class Addition
{
    public int Add(int a, int b)
    {
        return a + b;
    }
    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
}
```

★ Regeln

- Die Methoden die überladen werden sollen, müssen sich im Datentyp und/oder in der Anzahl der Übergabeparameter unterscheiden
- Der Ergebniswert hat auf die Überladung von Methoden keinen Einfluss

Statische Felder und Methoden

- ★ Für die Verwendung statischer Teile einer Klasse ist keine Instanz der Klasse notwendig
- ★ Wenn mehrere Instanzen einer Klasse erzeugt wurden und in jeder dieser Instanzen werden statische Daten verwendet, dann sind das immer dieselben Daten!
- ★ Statische Methoden heissen auch Klassenmethoden

- ★ Regeln
 - Innerhalb einer statischen Methode können Sie nur auf lokale und statische Variablen zugreifen.
 - In C# können Sie keine globalen Variablen anlegen. Wenn Sie diese Funktionalität brauchen, gehen Sie am besten wie folgt vor:
 - ★ Static Klasse anlegen (z.B. Global)
 - ★ private static Membervariablen für jede globale Variable
 - ★ Static Zugriffsmethoden für jede globale Variable

Beispiel statische Methode

```
public class Authenticator
{
    private static uint minPasswordLength = 6;
    ...
    public static uint GetMinPasswordLength()
    {
        return minPasswordLength ;
    }
    ...
}
```

★ Aufruf

```
uint len = Authenticator.GetMinPasswordLength();
```

Statische Klassen

- ★ Enthalten nur static members
- ★ Können nicht instanziiert werden
- ★ Sind sealed
- ★ Haben keinen Konstruktor

- Beispiele

```
public static class TempConverter
{
    public static double CtoF(double celsius)
    {
        return (celsius * 1.8) + 32; // Convert to Fahrenheit.
    }

    public static double FtoC(double fahrenheit)
    {
        return (fahrenheit - 32) / 1.8;
    }
}
```

Konstanten

Compile-Zeit Konstante const	Laufzeit-Konstante readonly
nur für primitive Datentypen, Enums und Strings	für alle Datentypen
muss bei der Deklaration initialisiert werden	kann auch im Konstruktor initialisiert werden
const double PI = 3.14159;	readonly int Start = 0;
wird bei der Compilation ersetzt	Compiler setzt Referenz auf Wert
sind immer statisch	können auch für Instanzen verwendet werden
	bevorzugt einsetzen

Konstruktoren

- ★ Beim Erzeugen eines Objekts aus einer Klasse mit dem Operator new wird der Konstruktor der entsprechenden Klasse aufgerufen
- ★ Der Konstruktor dient der vollständigen Initialisierung einer Klasse
- ★ Konstruktoren heissen gleich wie die Klasse
- ★ Konstruktoren haben keinen Returnwert

```
public class Rectangle
{
    private int _dx;
    private int _dy;

    // Constructor
    public Rectangle(int dx, int dy)
    {
        _dx = dx;
        _dy = dy;
    } ...
}
```

Gegenseitiger Aufruf von Konstruktoren

- ★ Konstruktoren können überladen werden und können sich gegenseitig aufrufen

```
public class Rectangle
{
    // Default Constructor (without arguments)
    public Rectangle() : this(0, 0)
    {
    }

    // Constructor with arguments
    public Rectangle(int dx, int dy)
    {
        _dx = dx;
        _dy = dy;
    } ...
}
```

Destruktoren / Finalizer

- ★ Im Destruktor werden die Aufräumarbeiten beim Löschen eines Objekts erledigt
- ★ Meistens ist es nicht nötig, in C# einen Destruktor zu erstellen, da dies der Garbage Collector erledigt
- ★ Destruktoren heissen gleich wie die Klasse mit einer vorangestellten ~, haben keinen Returnwert und keine Parameter

```
public class File
{
    // Destructor / Finalizer
    ~File()
    {
        // close file
    } ...
}
```


Namensräume

- ★ Ein Namensraum bezeichnet einen Gültigkeitsbereich für Klassen
- ★ In einem Namensraum können nur Klassen oder andere Namensräume deklariert werden, nicht jedoch Methoden oder Felder
- ★ Namensräume können verschachtelt werden

```
namespace CSharp
{
    // Deklarationen von Klassen und Namensräumen
    namespace Kurs
    {
        // Deklarationen für CSharp.Kurs
    }
}
```

Verwenden von Namensräumen

- ★ Fully qualified

```
CSharp.EineKlasse.EineMethode();
```

- ★ Import eines Namensraums

```
using CSharp;  
  
EineKlasse.EineMethode();
```

- ★ Bei Namensräumen wird häufig der Firmenname vorangestellt

```
namespace Microsoft.Excel.DataSheet
```

Der globale Namensraum

- ★ Alle Klassen, die nicht in einem angegebenen Namensraum deklariert werden, werden automatisch dem globalen Namensraum von C# zugewiesen
- ★ Der globale Namensraum ist immer vorhanden
- ★ Die Verwendung von eigenen Namenräumen sei hier ausdrücklich empfohlen

Grundlagen Datentypen

Ziel des Kapitels

- ★ Sie können mit den C#-Standard-Datentypen Variablen anlegen und die Variablen ineinander umwandeln.

Speicherverwaltung

★ Wertetypen

- Bei diesen Typen werden Werte direkt auf dem Stack gespeichert. Copy und Zuweisung by value

★ Referenztypen

- Speichert einen Verweis auf Daten. Werden auf dem Heap gespeichert. Copy und Zuweisung by reference

★ Stack

- Hier werden Daten solange abgelegt, wie sie tatsächlich gebraucht werden (z.B. für lokale Variablen oder Methodenparameter)
- Die Daten werden freigegeben, sobald sie nicht mehr benötigt werden

★ Heap

- Speicher auf dem Heap muss angefordert werden
- Das Löschen von auf dem Heap angelegten Objekten erledigt der Garbage Collector zu einem uns unbekannten Zeitpunkt

Garbage-Collection

- ★ Ist Bestandteil des Laufzeitumgebung
- ★ Räumt den Heap nach Bedarf und ohne Einfluss des Programmierers auf
- ★ Mehr dazu im Kurs «.NET-Framework-Grundlagen (CSA)»

Standard-Datentypen

★ sbyte

★ byte

★ char

★ short

★ ushort

★ int

★ UInt

★ long

★ ulong

★ float

★ double

★ decimal

★ bool

★ string

★ Erweiterte Datentypen (ab .NET 4.0)

- BigInteger: `BigInteger bigValue = BigInteger.Parse("98765432198765432");`
- Complex

Methoden von Datentypen

- ★ Alle Standarddatentypen sind Klassen
 - Sie haben nicht nur Daten, sondern auch Methoden
- ★ Alle Klassen sind von object abgeleitet
 - Über Vererbung stehen weitere Methoden zur Verfügung
- ★ Aufgabe
 - Studieren Sie in der Hilfe alle Members von Int32 und versuchen Sie, diese zu verstehen

Type und typeof()

- ★ Der Operator `typeof` wird wie eine Methode eingesetzt, ist jedoch Bestandteil der Sprache C#
- ★ Liefert beim Aufruf einen Wert vom Typ `Type`
- ★ Über Member-Variablen können Informationen über den Typ der entsprechenden Variablen ermittelt werden.

★ Beispiel:

```
class TestClass
{
    public static void Main()
    {
        int x = 200;
        Type t = typeof(Int32);

        if (t.Equals(x.GetType()))
            Console.WriteLine("x ist vom Typ Int32.");
        else
            Console.WriteLine("x ist nicht vom Typ Int32.");
    }
}
```

Konvertierung (1/2)

★ Implizite Konvertierung

- Wird beispielsweise eine initialisierte Variable vom Typ byte einer int-Variable zugewiesen, wird eine automatische (oder eben implizite) Typkonvertierung durchgeführt

```
int i;  
byte b = 100;  
i = b;           // implizite Konvertierung von byte in int
```

Konvertierung (2/2)

★ Explizite Konvertierung

- Auch als Casting bezeichnet
- Die explizite Konvertierung müssen Sie immer dann einsetzen, wenn der Zieldatentyp kleiner als der Stammdatentyp ist. Bei einer expliziten Konvertierung sind Sie als Programmierer dafür verantwortlich, dass die Konvertierung erfolgreich durchgeführt werden kann
- Der gewünschte Datentyp wird in Klammern vor den zu konvertierenden Wert oder Ausdruck gesetzt.

★ Beispiel:

```
int i = 100;  
byte b;  
b = (byte)i;    // explizite Konvertierung
```

★ Was passiert, wenn i 400 statt 100 ist?

- b wird dann zu 144 -> Überlauf

checked

- ★ Die Konvertierung wird nun innerhalb des checked-Blocks überwacht. Sollte sie fehlschlagen, wird eine Exception ausgelöst (hier eine `System.OverflowException`), die Sie abfangen können

```
byte target;  
int source = 400;  
  
checked  
{  
    target = (byte)source;  
    Console.WriteLine("Wert: {0}", target);  
}
```

as Operator

★ Alternative zur expliziten Konvertierung

```
object o = Factory.GetObject();  
MyType t = o as MyType;  
if (t != null)  
    // arbeite mit t, es ist ein MyType.  
else  
    // Typkonvertierung nicht erfolgreich.
```

Umwandlungsmethoden der Klasse Convert

- ★ ToBoolean()
- ★ ToByte()
- ★ ToChar()
- ★ ToDateTime()
- ★ ToBoolean()
- ★ ToDate()
- ★ ToDecimal()
- ★ ToDouble()
- ★ ToInt16()
- ★ ToInt32()
- ★ ToInt64()
- ★ ToSByte()
- ★ ToSingle()
- ★ ToString()
- ★ ToUInt16()
- ★ ToUInt32()
- ★ ToUInt64()

```
string input = "4711";  
int number = Convert.ToInt32(input);
```

Boxing und Unboxing

- ★ Ermöglicht dem Programmierer (nahezu) den gleichen Umgang mit Werte- und Referenztypen
- ★ Wenn ein Wertetyp als Referenztyp verwendet werden soll, werden die enthaltenen Daten «verpackt». C# benutzt dafür den Datentyp object, der bekanntlich die Basisklasse aller Datentypen darstellt, das bedeutet auch: jeden Datentyp aufnehmen kann
- ★ Object merkt sich, welche Art von Daten in ihm gespeichert wurden, damit auch die Rückwandlung möglich ist

Unboxing

★ Ein Referenztyp wird in einen Wertetyp verwandelt

```
int i = 100;
```

```
object o;
```

```
o = i; // Boxing !!
```

```
//Rückkonvertierung (Unboxing)
```

```
byte b = (byte)(int)o; // funktioniert!!
```

Strings

- ★ Der Datentyp String ist universell einsetzbar
- ★ Obwohl die Deklaration wie bei einem Wertetyp funktioniert, handelt es sich bei einem string immer um einen Referenztyp
- ★ Ein String ist bezüglich der Grösse dynamisch. Das heisst, er nimmt sich vom Heap soviel Speicher, wie er gerade braucht
- ★ Strings in C# sind immer Unicode

Stringzuweisungen

★ direkte Zuweisung:

```
string myString = "Hallo Welt";
```

★ oder:

```
string myString;  
myString = "Hallo Welt";
```

★ Zuweisung über die Copy() – Methode:

```
string myString = "Bill Gates";  
string myString2 = string.Copy(myString);
```

Aufgabe

★ Studieren Sie in der Online-Hilfe die Methoden, die string zur Verfügung stellt

★ Prüfung auf null oder Leerstring:

```
if (myStr != null || myStr.Length > 0) ...
```

```
if (!string.IsNullOrEmpty(myStr)) ...
```

```
if (!string.IsNullOrWhiteSpace(myStr)) ...
```

Formatierung

- ★ Die Angabe, welche Art von Formatierung gewünscht wird, geschieht im Platzhalter durch die Angabe eines Formatzeichens und ggf. einer Präzisionsangabe für die Anzahl Stellen, die ausgegeben werden sollen

Beispiel

```
Console.Write("Geben Sie Zahl 1 ein: ");  
int a = Convert.ToInt32(Console.ReadLine());  
Console.Write("Geben Sie Zahl 2 ein: ");  
int b = Convert.ToInt32(Console.ReadLine());  
Console.WriteLine("Die Zahlen lauten:");  
Console.WriteLine("Zahl 1: {0:D5}",a);  
Console.WriteLine("Zahl 2: {0:D5}",b);
```

Ausgabe:

Zahl 1:00075

Zahl 2:01024

Selbst definierte Formate

- ★ Ausgabe einer Zahl im Währungsformat, negative Zahlen in Klammern, mit Tausendertrennzeichen

```
using System;
```

```
public class Beispiel
```

```
{
```

```
    public static void Main()
```

```
    {
```

```
        Console.Write("Geben Sie eine Zahl ein: ");
```

```
        int a = Convert.ToInt32(Console.ReadLine());
```

```
        Console.WriteLine("{0:#,#.00' Fr';(#,#.00)' Fr'}",a);
```

```
    }
```

```
}
```

Aufgabe

- ★ Studieren Sie die Formatierungsmöglichkeiten in den Kursunterlagen!

Übungen

- ★ Bearbeiten Sie die Übungen im Stoff und spielen Sie mit den verschiedenen Formatierungsmöglichkeiten

Ablaufsteuerung

Ziel

- ★ Sie setzen die verschiedenen Befehle der Ablaufsteuerung ein
 - Sprünge: goto, break, continue
 - Bedingungen: if, switch, ?
 - Schleifen: for, while, do – while, foreach

Absolute Sprünge

- ★ Diese sollen hier lediglich der Vollständigkeit halber aufgeführt werden. Absolute Sprünge (Stichwort goto) gelten als schlechter Programmierstil!

```
goto Labelbezeichner;  
    // Anweisungen  
Labelbezeichner:  
    // Anweisungen
```

Bedingungen und Verzweigungen

==	Vergleich auf Gleichheit
!=	Vergleich auf Ungleichheit
>	Vergleich auf grösser
<	Vergleich auf kleiner
>=	Vergleich auf grösser oder gleich
<=	Vergleich auf kleiner oder gleich

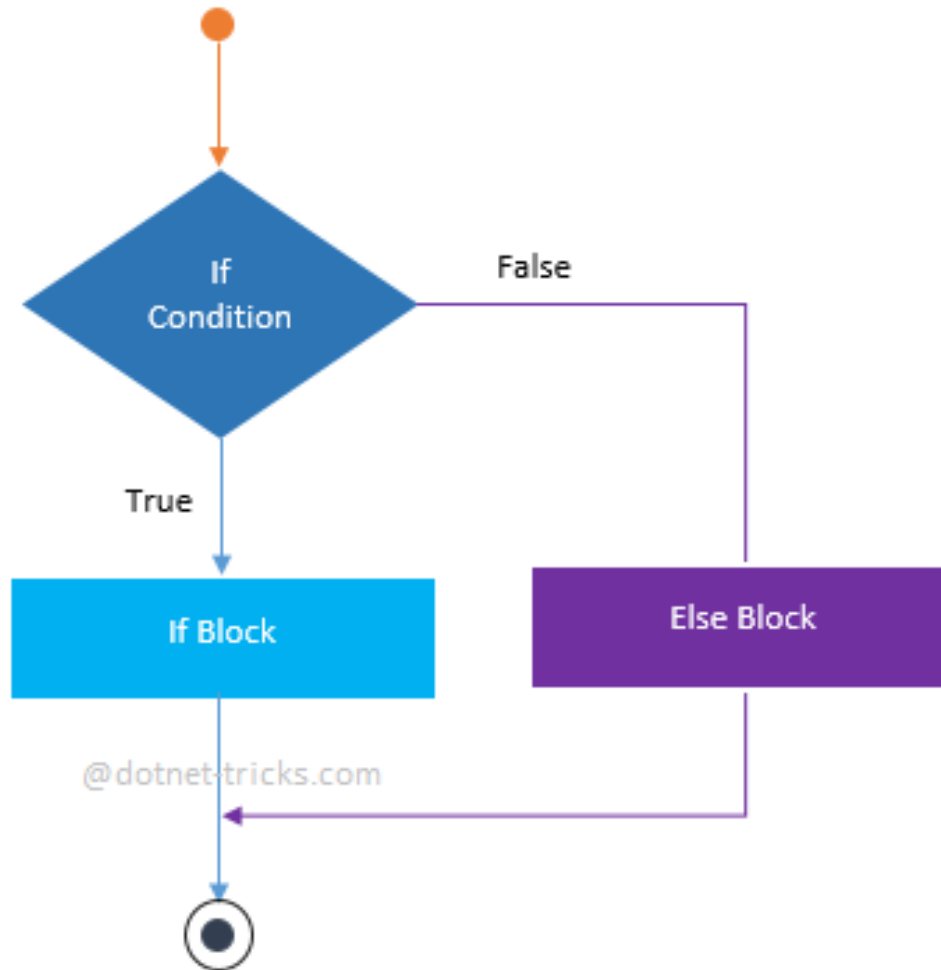
Logische Vergleiche

!	nicht Operator (aus wahr wird falsch und umgekehrt)
&&	und – Verknüpfung (beide Bedingungen müssen wahr sein)
	oder – Verknüpfungen (eine der Bedingungen muss wahr sein)

Die if-Anweisung

```
if (Bedingung)
{
    // Anweisungen, wenn Bedingung wahr
}
else if (Bedingung)
{
    // Anweisungen, wenn else if Bedingung wahr
}
else
{
    // Anweisungen, wenn Bedingung falsch
}
```

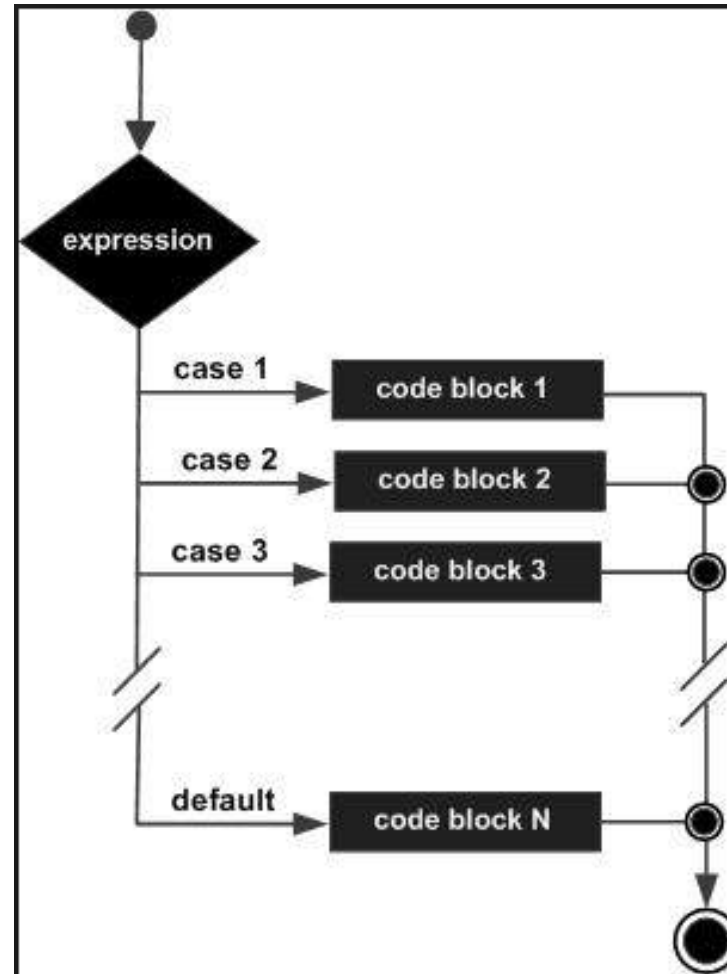
Ablauf



Die switch-Anweisung

```
switch (variable)
{
    case 1:
        // Anweisungen
        break;
    case 2:
        // Anweisungen
        break;
    default:
        // Standard-Anweisungen
        break;
}
```

Ablauf



Die bedingte Zuweisung

★ Bedingung ? Ausdruck1 : Ausdruck2;

```
if (Bedingung)
{
    Ausdruck1;
}
else
{
    Ausdruck2;
}
```

★ Die bedingte Zuweisung stellt eine Schreiberleichterung dar. Doch es besteht die Gefahr, dass es für ungeübte Programmierer unübersichtlich wird.

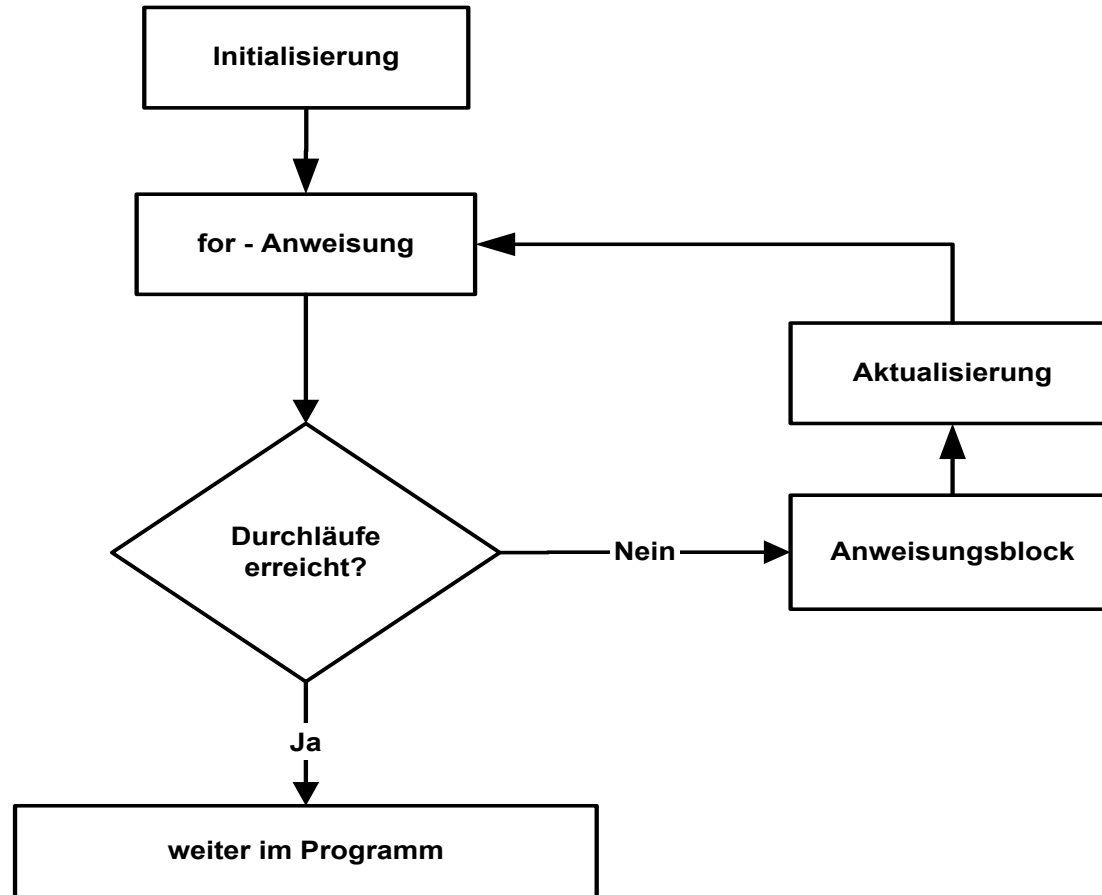
```
public int Max(int a, int b)
{
    return a > b ? a : b;
}
```

Die for-Schleife

```
for (Initialisierung; Bedingung; Aktualisierung)
{
    // Anweisungen
}
```

- ★ Sie wird üblicherweise benutzt, wenn die Anzahl der Schleifendurchläufe bekannt ist

Ablauf



Beispiel

```
for (int i = 1; i <= 10; ++i)
{
    System.Console.WriteLine(i);
}
```

★ Was wird hier ausgegeben?

Schleifenbeeinflussung

- ★ break

Mit der Anweisung break brechen Sie die Ausführung des aktuellen Programmblockes ab, d.h. die Schleife bricht ab

- ★ continue

Mit dieser Anweisung beginnt die Schleife von vorn, ohne dass die Anweisungen nach continue ausgeführt werden. Die Schleifenvariable wird dabei entsprechend der Aktualisierung verändert

Die while-Schleife

- ★ Damit werden Schleifen programmiert, die nicht unbedingt durchlaufen werden müssen (abweisende Schleife)

```
while (Bedingung)
{
    // Anweisung
}
```


Die do-while-Schleife

- ★ Schleife wird mindestens einmal durchlaufen, da die Bedingung erst am Ende des Programmblocks überprüft wird

```
do
{
    // Anweisungen
} while (Bedingung);
```

Operatoren

Ziel

- ★ Sie wissen, welche Operatoren C# anbietet
- ★ Sie wissen, wie Sie diese Operatoren benutzen

Mathematische Operatoren

★ C# stellt die folgenden mathematischen Operatoren zur Verfügung:

+	Addition
-	Subtraktion
*	Multiplikation
/	Division
%	Modulo (Rest)
++	Inkrement
--	Dekrement

Zusammengesetzte Rechenoperatoren

+=	Addition und gleichzeitige Zuweisung $x += y$; entspricht $x = x + y$;
-=	Subtraktion und gleichzeitige Zuweisung $x -= y$; entspricht $x = x - y$;
*=	Multiplikation und gleichzeitige Zuweisung $x *= y$; entspricht $x = x * y$;
/=	Division und gleichzeitige Zuweisung $x /= y$; entspricht $x = x / y$;
%=	Modulo und gleichzeitige Zuweisung $x \% = y$; entspricht $x = x \% y$;

Die Klasse Math

- ★ Die Klasse Math ist im Namensraum-System implementiert und stellt einige statische, mathematische Methoden und Konstanten zur Verfügung

Aufgabe

★ Schauen Sie sich die Klasse Math und vor allem die Eigenschaften der Klasse an

Mathematische Operatoren

```
using System;
public class Converter
{
    public static double DegToRad(double a)
    {
        return (a / 180 * Math.PI);
    }

    public static double RadToDeg(double a)
    {
        return (a * 180 / Math.PI);
    }
}
```


Bitweise Operatoren

&	Bewirkt eine bitweise UND-Verknüpfung zweier Werte
	Bewirkt eine bitweise ODER-Verknüpfung zweier Werte
^	Bewirkt eine bitweise exklusiv-ODER-Verknüpfung zweier Werte
>>	Bewirkt eine Verschiebung aller Bits um n Stellen nach rechts
<<	Bewirkt eine Verschiebung aller Bits um n Stellen nach links

Erweiterte Datentypen: Arrays, Structs, Enumerations

Ziele

- ★ Wir erarbeiten uns in diesem Kapitel die Kenntnisse über:
 - Arrays
 - Structs
 - Aufzählungstypen

Arrays

- ★ Deklaration:

```
int[] intArray = new int[10];
```

- ★ Zugriff auf ein Element:

```
intArray[2] = 115;
```

- ★ Alle Arrays sind implizit von der Klasse System.Array abgeleitet

Wichtig

- ★ Die Nummerierung des Indexes von C# Arrays beginnt immer mit 0
- ★ Gültige Werte für den Index für den hier definiertem Array iArray sind somit 0 bis 9

```
int[] intArray = new int[10];
```

Mehrdimensionale Arrays

★ Deklaration:

```
int[,] theMatrix = new int[10, 5];
```

★ Dieses Array hat 10 Zeilen und 5 Spalten

Ungleichförmige Arrays (Jagged Arrays)

★ Deklaration:

```
int[][] jaggedArray = new int[4][];  
jaggedArray[0] = new int[8];  
jaggedArray[1] = new int[4];  
jaggedArray[2] = new int[2];  
jaggedArray[3] = new int[4];
```

Arrays initialisieren

```
int[] theArray = {15, 17, 19, 21, 23};
```

```
int[,] the2Darray = {{1, 1}, {2, 2}, {3, 3}};
```

★ Sehr häufig werden Arrays innerhalb von Schleifen initialisiert

Die foreach-Schleife

- ★ Diese spezielle Schleife ermöglicht uns, alle Elemente einer Liste zu durchlaufen
- ★ Die Elemente stehen bei dieser Vorgehensweise nur read only zur Verfügung

```
foreach (Datentyp Bezeichner in Liste)
{
    // Anweisungen
}
```

Beispiel foreach

```
int[] theArray = new int[10];
```

```
for (int i = 0; i < 10; i++)  
    theArray[i] = i;
```

```
foreach (int u in theArray)  
{  
    if (u < 5)  
    {  
        Console.WriteLine(u);  
    }  
}
```

Structs

- ★ Structs sind C# Strukturen, die sowohl Daten als auch Methoden enthalten können. Sie werden genau wie Klassen deklariert, können auch Konstruktoren besitzen und werden mit derselben Syntax beschrieben
- ★ Structs sollen dort eingesetzt werden, wo kleine Objekte in grosser Menge innerhalb einer Applikation verwendet werden
- ★ Structs unterstützen keine Vererbung

Structs repräsentieren Wertetypen

Beispiel struct

```
public struct Coordinate  
{  
    public int x;  
    public int y;  
}
```

★ Faustregel

- Ein struct ist dann effizienter als eine Klasse, wenn seine Grösse weniger als 16 Byte beträgt

Aufzählungstyp

- ★ Alle Elemente von Aufzählungstypen (enum) besitzen den gleichen Wertetyp

```
[Modifikator] enum Bezeichner [:Typ]
{
    Wert_1, Wert_2, Wert_3, ..., Wert_n
};
```

Beispiel Standardaufzählung

```
public enum WeekDays  
{  
    Sonntag,  
    Montag,  
    Dienstag,  
    Mittwoch,  
    Donnerstag,  
    Freitag,  
    Samstag  
};
```

Anpassen der Werte

```
public enum Months : byte
{
    Januar = 31,
    Februar = 28,
    März = 31,
    ...
};
```

Flag Enums

```
[Flags]
public enum PrinterSettings
{
    PrintPageNr          =      0x00000001,
    PrintHeader          =      0x00000002,
    PrintFooter          =      0x00000004,
    PrintTitle           =      0x00000008,
    PrintGraphics        =      0x00000010
};
```


Einsatz

★ Werte eines Enum Flags können miteinander kombiniert werden:

```
case 1: return PrintPageNr;  
case 2: return PrintHeader | PrintFooter;  
...
```

Vererbung und Interfaces

Ziele

- ★ Wir betrachten in diesem Kapitel die C#-Funktionalitäten:
 - Klassenvererbung
 - Interfaces (Schnittstellenvererbung)

Vererbung von Klassen

- ★ Bei der Vererbung wird eine Klasse von einer bestehenden Klasse abgeleitet und kann dann erweitert werden. Die abgeleitete Klasse erbt dabei die Eigenschaften und Funktionalitäten der Elternklasse
- ★ Eine Klasse kann in C# beliebig viele Nachfolger, aber nur einen Vorgänger haben

- ★ Syntax

```
[Modifikator] class Klasse : Elternklasse  
{  
    // Felder und Methoden  
}
```

Zugriffsmodifikatoren

Modifizier	Zugriff aus anderen Klassen	Zugriff aus abgeleiteter Klasse	Zugriff in eigener Klasse	Zugriff innerhalb von Assembly
public	x	x	x	x
protected	-	x	x	-
private	-	-	x	-
internal	-	-	x	x
internal protected	-	x	x	x

Polymorphismus (Mehrgestaltigkeit)

- ★ Wird eine Klasse von einer anderen Klasse abgeleitet, so kann eine Instanz der neuen Klasse auch der Elternklasse zugewiesen werden
- ★ Verschiedene Objekte reagieren auf den Aufruf einer gleichen Methode unterschiedlich

Überschreiben von Methoden

★ In der Basisklasse

```
public virtual string GetAdr()  
{  
    return string.Format("{0}\n{1}\n{2}", name, strasse, ort);  
}
```

★ In der abgeleiteten Klasse

```
public override string GetAdr()  
{  
    string x = base.GetAdr();  
    return x + string.Format("\n{0}", tel);  
}
```

Den Basis-Konstruktor aufrufen

- ★ Der Konstruktor einer Klasse kann nicht vererbt werden
- ★ Jede Klasse benötigt einen oder mehrere Konstruktoren
- ★ Der Konstruktor der Basisklasse wird wie folgt aufgerufen:

```
public SpezialAdresse() : base()  
{  
    // Basis-Konstruktor wird  
    // automatisch aufgerufen  
}
```

- ★ Bei Bedarf können auch Argumente übergeben werden

Regel: Jede Klasse initialisiert die eigenen Felder

Abstrakte Klassen

Für abstrakte Klassen gilt:

- ★ Es gibt nie eine Instanz dieser Klasse
- ★ Abstrakte Methoden müssen in abgeleiteten Klassen überschrieben werden.

```
public abstract class Arbeitnehmer
{
    protected string name;
    protected string vorname;

    public abstract string GetBeruf();

    public string GetName()
    {
        return string.Format("{0}, {1}", name, vorname);
    }...
}

public class Elektriker : Arbeitnehmer
{
    public override string GetBeruf() { return "Elektriker"; }
}
```

Verbergen von Methoden

- ★ Wenn Sie von einer Klasse ableiten und eine Methode überschreiben, die in der Elternklasse nicht virtuell oder abstrakt ist, müssen Sie den Compiler anweisen, die Methode der Basisklasse zu verbergen. Das reservierte Wort dazu heisst `new`
- ★ Beispiel

```
class DerivedAddress : BaseClass
{
    new public string GetAdr() // nicht empfohlen !!
    {
        return string.Format("{0}, {1} {2}", name, plz, ort);
    }
}
```

Unterschied virtuelle vs. verborgene Methoden

★ Virtuelle Methoden

- Beim Aufruf einer Methode über eine Referenz vom Typ der Basisklasse auf ein Objekt der abgeleiteten Klasse wird die Methode der abgeleiteten Klasse aufgerufen (was auch zu erwarten ist)

★ Verborgene Methoden

- Beim Aufruf einer Methode über eine Referenz vom Typ der Basisklasse auf ein Objekt der abgeleiteten Klasse wird die Methode der Basisklasse aufgerufen
- Die Verwendung von new gilt als Hack und darf nur ausnahmsweise eingesetzt werden, wenn die Basisklasse nicht angepasst werden kann

Versiegelte Klassen

- ★ Man kann in C# auch Klassen definieren, von denen keine Klassen abgeleitet werden können
- ★ Ein Beispiel einer versiegelten Klasse haben wir in Form der string-Klasse bereits kennengelernt. Es ist somit nicht möglich, eine eigene Klasse von string abzuleiten und sie zu erweitern

```
public sealed class Adresse
{
    private string name;
    ...
}
```

- ★ Versiegelte Klassen dürfen keine abstrakten Methoden enthalten
- ★ Nicht erlaubt sind sämtliche Modifikatoren, die mit Vererbung zu tun haben wie z.B. virtual

Interfaces

- ★ Interfaces (Schnittstellen) verhalten sich ähnlich wie abstrakte Klassen. Es handelt sich dabei um die Deklaration verschiedener Methoden, die aber nie eine Implementation enthalten
- ★ Ein wichtiger Unterschied bei der Anwendung von abstrakten Klassen und Interfaces ist der, dass Interfaces eine Mehrfachvererbung ermöglichen

Beispiel Interface

```
public interface IGeometric
{
    double GetArea();
    double GetPerimeter();
}
```

```
public class Rechteck : GeoClass, IGeometric
{
    public double SeiteA = 0;
    public double GetArea() { ... }
    public double GetPerimeter() { ... }
    ...
}
```

Aufgabe

- ★ Studieren Sie den im Stoff abgedruckten Code des Beispiels «Interface»
- ★ Studieren Sie das mitgelieferte Beispiel «Interface 2», das die Mehrfach-Vererbung von Interfaces zeigt

Explizite Interfaces

```
public class Rechteck : GeoClass, IGeometric
{
    // Implementation der Interface-Methoden
    double IGeometric.GetArea()
    {
        return seiteA * seiteB;
    }
    double IGeometric.GetPerimeter()
    {
        return 2 * seiteA + 2 * seiteB;
    }
}
```


Eigenschaften und Indexer

Ziel

- ★ Sie können Eigenschaften (auch Properties genannt) von Klassen anlegen und wissen, wann sie eingesetzt werden

Eigenschaften

- ★ Für den Programmierer einer Anwendung verhalten sich Eigenschaften wie Felder einer Klasse, deren Werte geschrieben und/oder ausgelesen werden können
- ★ Der Unterschied besteht darin, dass Eigenschaften für das Lesen und Schreiben der Werte Methoden verwenden
- ★ Vorteile gegenüber public-Feldern:
 - Unterscheidung lesender oder schreibender Zugriff
 - Breakpoints setzen
 - Bereichsüberprüfung
 - Überschreiben möglich

Beispiel Properties

```
public class Artikel
{
    private string _name = "Peter";
    private double _price = 0.0;
    public string Name
    {
        get { return _name; }
        set { _name = value; }
    }
    ...
    Artikel myArtikel = new Artikel("noName", 0);
    myArtikel.Name = Console.ReadLine();
    ...
}
```

Erweiterungen Properties

★ Unterschiedliche Zugriffsberechtigung (C# 2.0)

```
public string Name
{
    get { return _name; }
    protected set { _name = value; }
}
```

★ Abgekürzte Schreibweise (C# 3.0)

```
public string FirstName { get; set; }
```

Indexer

```
public class Sentence
{
    private string [] words = "an array of words".Split(' ');

    public string this[int wordNum] // indexer
    {
        get { return words[wordNum]; }
        set { words[wordNum] = value; }
    }
}
```

★ Anwendung

```
Sentence s = new Sentence();
String word = s[0]; // 'an,
s[1] = 'stringarray';
```

Strukturierte Fehlerbehandlung

Ziele

- ★ Sie kennen die Bedeutung von strukturiertem Exception Handling
- ★ Sie können in C#
 - Exceptions auslösen
 - Exceptions abfangen
 - Unmanaged Ressourcen korrekt verwenden

Exceptions

- ★ Exceptions (Ausnahmen) sind vom Framework ausgelöste Ausnahmeverfahren. Man kann sich Exceptions etwa wie Meldungen vorstellen, die vom Framework an die Applikation gesendet werden
- ★ Die Applikation fängt diese Meldungen (hoffentlich) und kann dann angemessen reagieren

Exceptions abfangen

```
try
{
    return a / b;
}
catch (Exception e)
{
    Console.WriteLine("Ausnahmefehler: {0}", e.Message);
}
```

Beschreibung

- ★ Im catch-Block werden die Exceptions abgefangen, die von den Anweisungen innerhalb des vorstehenden try-Blocks auftreten
- ★ Tritt tatsächlich eine Exception auf, werden die noch nicht behandelten Anweisungen nicht mehr aufgerufen

try – catch – finally

```
try
{
    return a / b;
}
catch (DivideByZeroException e)
{
    Console.WriteLine("Division durch 0 ist verboten");
}
catch (Exception e)
{
    Console.WriteLine("Ausnahmefehler: {0}", e.Message);
}
finally
{
    // wird immer ausgeführt
}
```

Wichtig

- ★ Alle Exceptions werden irgendwo gefangen. Wenn Sie es nicht in Ihrem Programm tun, macht dies das Framework für Sie. Dies bedeutet jedoch im Allgemeinen ein mehr oder minder brüskes Runterfahren Ihrer Applikation

Exceptions auslösen

- ★ Exceptions auslösen (auch werfen genannt), wird mit dem reservierten Wort throw erreicht
- ★ Alle Exceptions im .NET Framework sind von der Klasse Exception abgeleitet
- ★ C# ermöglicht Ihnen, eigene Exception Klassen anzulegen. Sie müssen dazu Ihre Klasse ebenfalls von der Klasse Exception ableiten

- ★ Beispiele:

```
throw new ApplicationException();
```

```
throw new IndexOutOfRangeException("idx=" + index.ToString());
```

Anwendungstipps

- ★ Nur einen try-Block pro Methode
- ★ Exceptions so spezifisch wie möglich fangen
 - Z.B. `FileNotFoundException` statt `IOException`
- ★ Fangen Sie Exceptions nur, wenn Sie auch etwas Sinnvolles damit anfangen können
- ★ using anstelle von `try` – `finally` verwenden

Wie weiter

.NET-Framework-Grundlagen

- ★ Komponenten + Assemblies
- ★ Effektive Programmierung (StringBuilder + Collection Klassen)
- ★ Generics und Generic Collections
- ★ LINQ (Language Integrated Query)
- ★ Attribute
- ★ Delegates und Ereignisse
- ★ Interoperabilität mit unverwaltetem Code
- ★ Garbage Collection
- ★ Serialisierung
- ★ Ein- und Ausgabe
- ★ Datenaustausch mit XML
- ★ Debugging + Tracing